

MCP Server Curriculum

Part of the [training document series](#). This folder is a self-contained curriculum for building an MCP server — a standard-protocol layer that exposes an existing data source or API as tools an LLM can call, sitting between the [API backend](#) (or any other data source) and the [conversational agent](#) that will use it.

Reading order

1. **Core Concepts** — what MCP is and the problem it solves, the host/client/server roles, the three primitives (tools, resources, prompts), transports, and a minimal working server.
2. **Tool Design** — designing tool schemas for an LLM audience: direct wraps vs. composed tools, argument-resolution guards, shaping responses for a model rather than a human, error handling, statelessness, interpretive framing, envelope pass-through with computed summaries, and rejecting what a tool cannot honor.
3. **Testing** — unit-testing tool functions directly, integration-testing through a real client, fixture data patterns, a class of entry-point bug only a real subprocess launch can catch, and registry-wide tests for rejection rules.
4. **Deployment** — transports in production, per-boundary auth decisions (a worked four-boundary example with edge service tokens), connecting local, desktop-app and programmatic clients, and independent versioning.

Prerequisites

Comfort with Python functions, classes, and type hints is assumed. Having read the [Python API Backend curriculum](#) first is helpful but not required — an MCP server commonly wraps a REST API like the one built there, and several documents in this folder point back to the equivalent backend concept for comparison.

See also

- [Architecture Overview](#) for how this layer fits between a data-serving API and an LLM agent
- [Python API Backend](#) for a typical thing this kind of server wraps
- [Conversational Agents](#) for the typical consumer of an MCP server's tools